

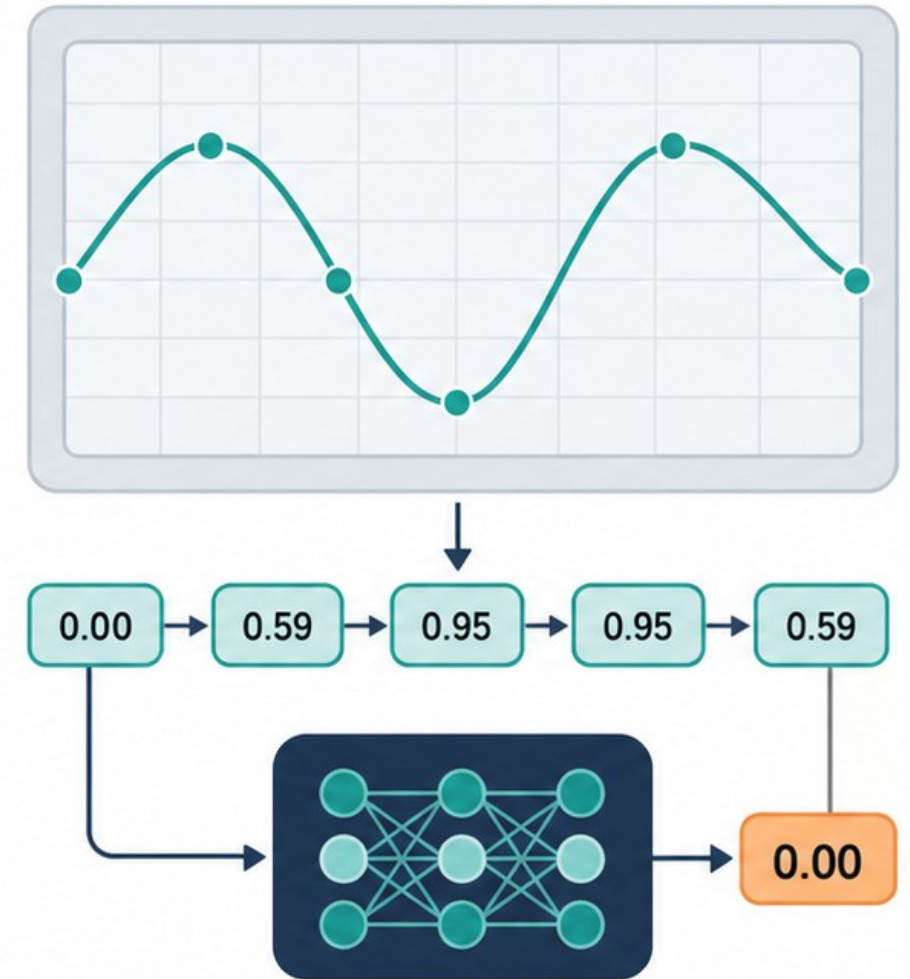
Large Language Models for Measurement & Automation

How an LLM sees a sine wave, and how to put one to work in the lab

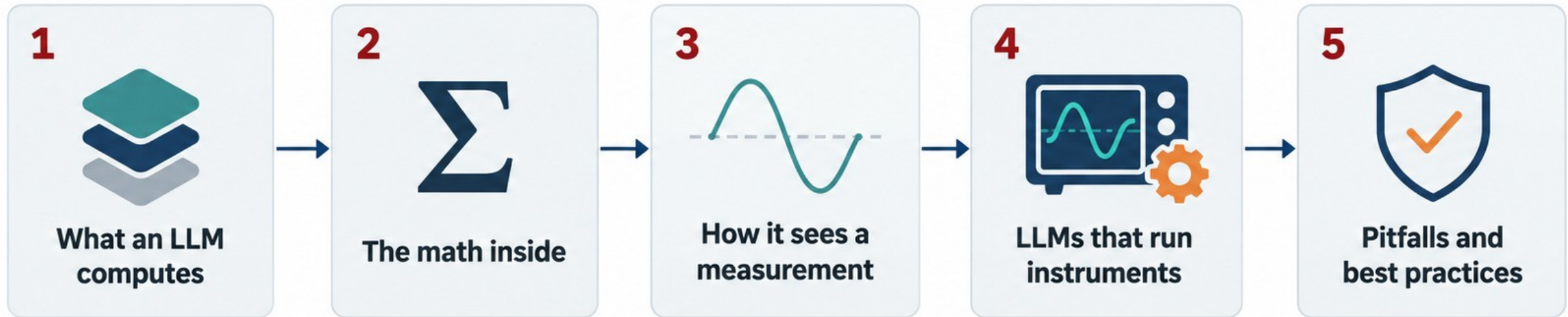
Arash Ahmadi

Ph.D. Student, Electrical and Computer Engineering
INQUIRE Lab, The University of Oklahoma

Guest lecture | Dr. Sarah Sharif's class

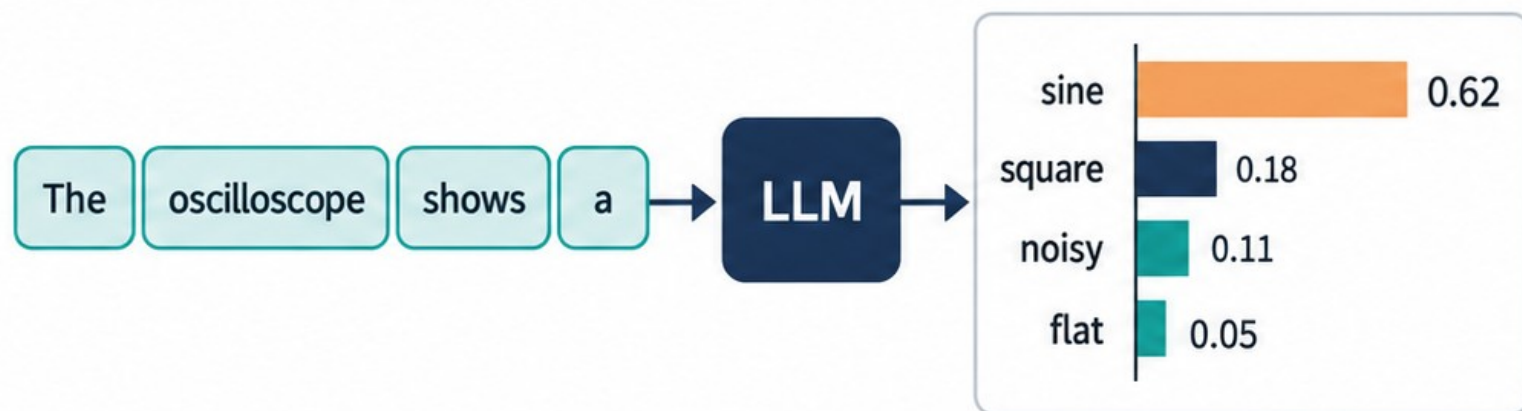


Today's roadmap



Goal: know when to trust an LLM with your data, and how to wire one into a measurement system safely.

An LLM is a next-token predictor



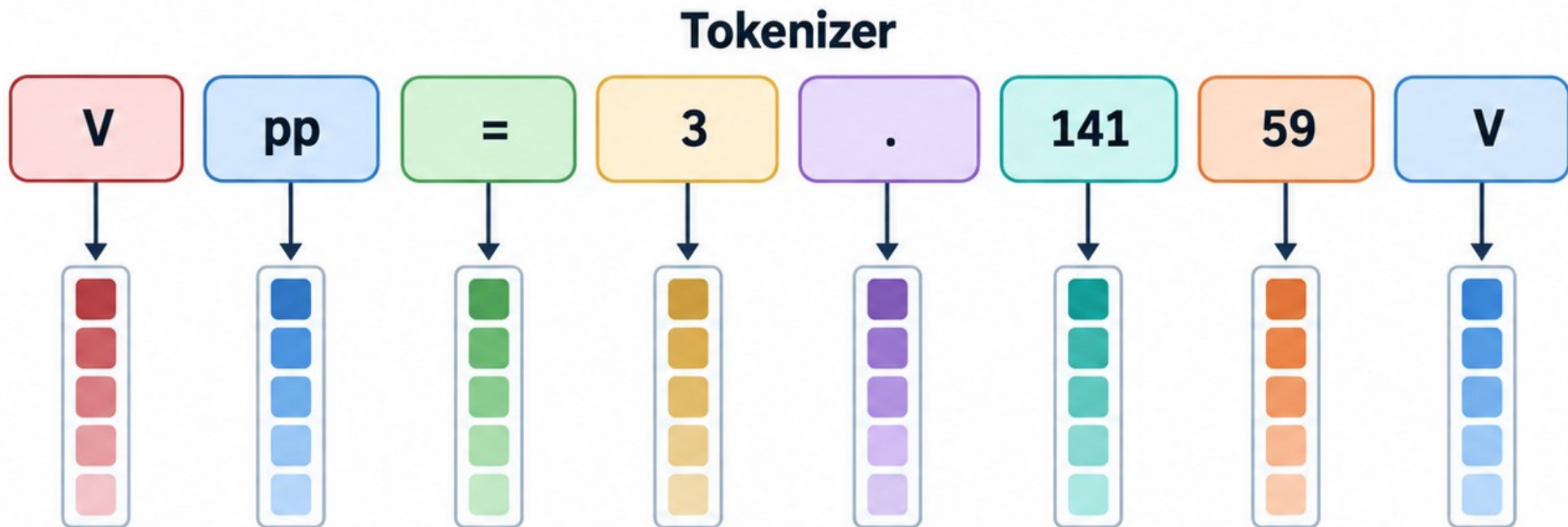
$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$$

$$x_t \sim \text{softmax}(z_t / \tau)$$

One token at a time. Temperature τ controls randomness.

Everything an LLM does, from chat to writing lab code, is this loop repeated.

Step 1: Text becomes tokens, tokens become vectors



Embedding $e_i = W_E[token_i] \in \mathbb{R}^d$

Numbers are split into arbitrary pieces: $3.14159 \rightarrow '3' '.' '141' '59'$. The model never sees the value, only the pieces.

Position is added so order matters:
 $h_i = e_i + p_i$

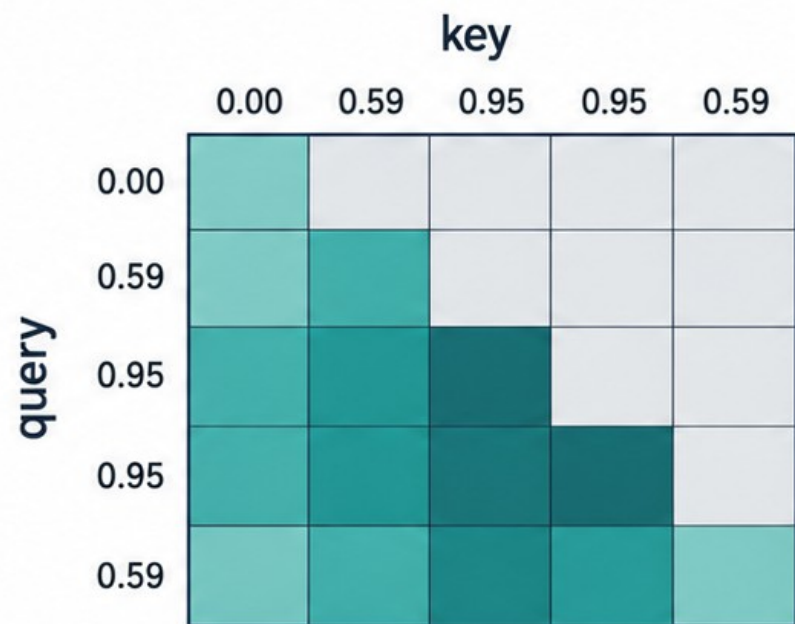
Step 2: Attention, the core equation

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

$Q = HW_Q$ (what I am looking for)

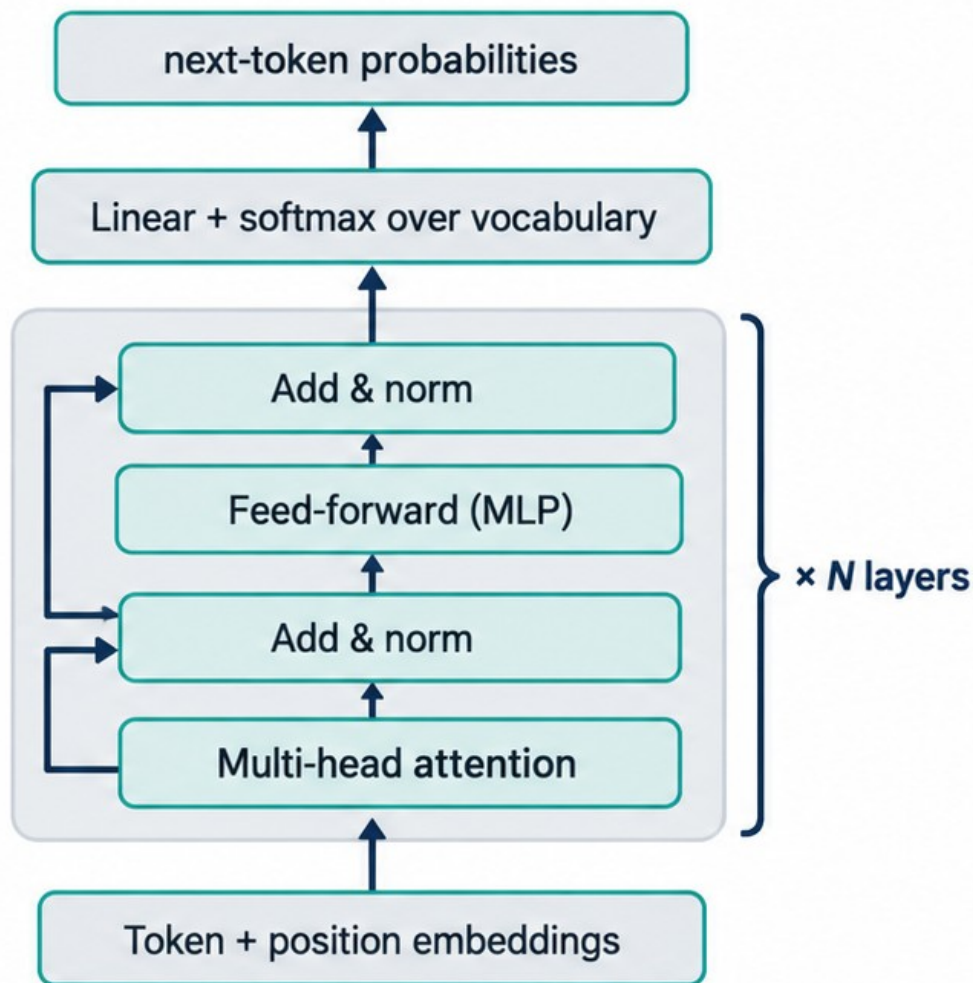
$K = HW_K$ (what I contain)

$V = HW_V$ (what I pass on)



Each new sample looks back at earlier samples. The causal mask hides the future.

Step 3: Stack it, then learn it



Training objective

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t})$$

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

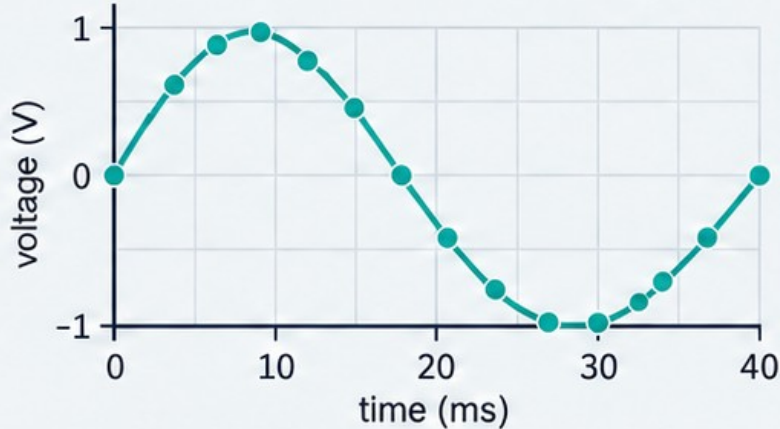
- Cross-entropy on trillions of tokens
- Billions of parameters θ
- Same math whether the text is prose, code, or numbers

How an LLM sees a measurement: the sine wave

What the scope sees

$$v(t) = A \sin(2\pi ft + \phi)$$

sampled at f_s : $v[n] = v\left(\frac{n}{f_s}\right)$



What we send

0.00, 0.59, 0.95, 0.95, 0.59,
0.00, -0.59, -0.95, -0.95,
-0.59, 0.00, ...

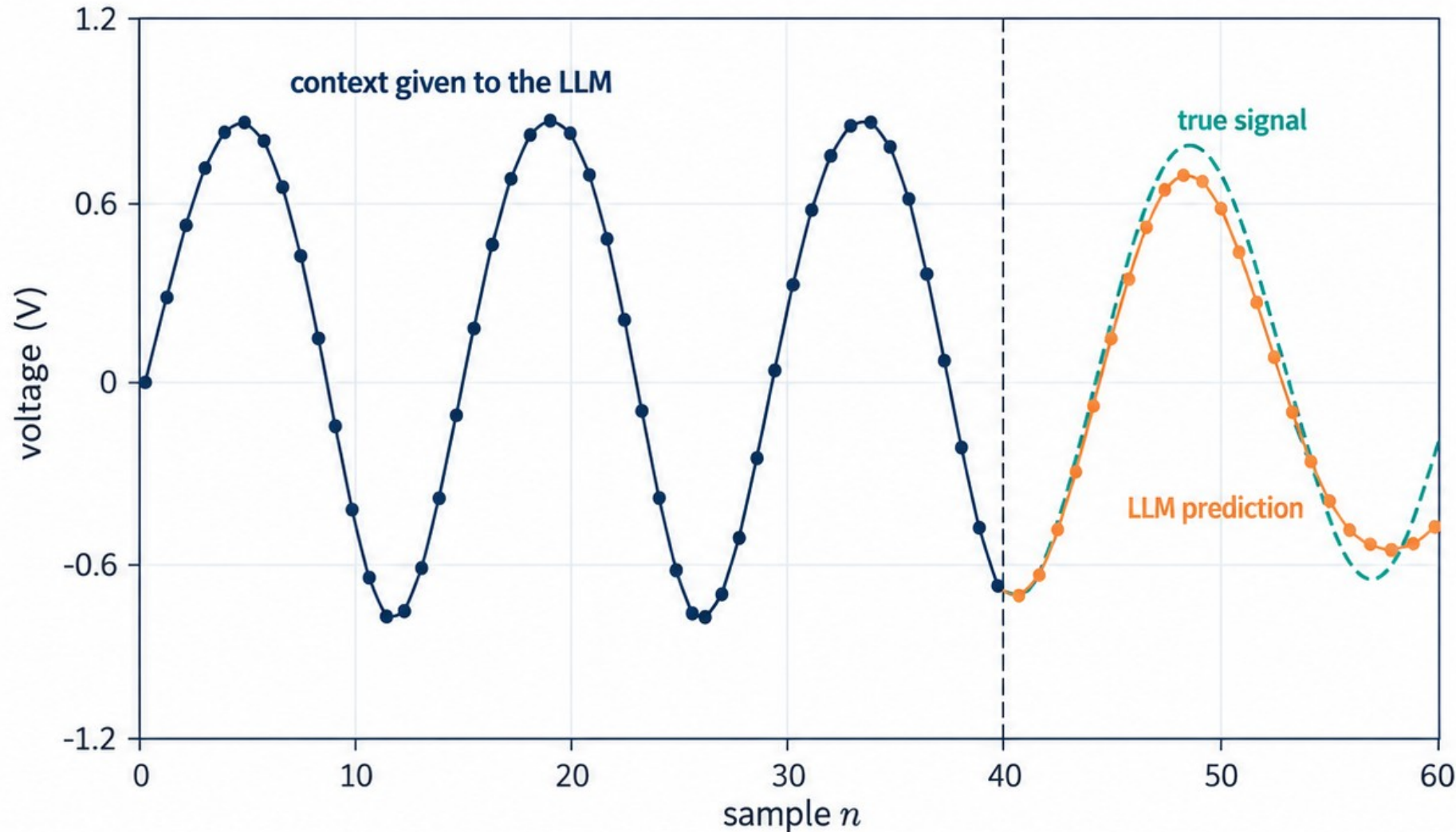
What the LLM actually gets

0	.	00	,	0	.	59	,
0	.	95	,	0	.	95	,
0	.	59	,	0	.	00	,
-	0	.	59	,	-	0	.
95	,	-	0	.	95	,	...

a string of tokens, not a waveform

The model has no concept of volts, time, or frequency. It only sees a pattern in text.

In-context learning: the LLM continues the wave



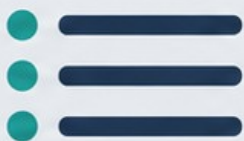
Works for clean, periodic data: the pattern is in the text.

Errors grow with horizon: small phase and amplitude drift.

Noise, drift and long records quickly exceed what text can carry.

Idea from Gruver et al., 'Large Language Models Are Zero-Shot Time Series Forecasters', NeurIPS 2023. Illustration.

Four ways to give measurements to a model



1 Serialize as text

Fixed precision, scaled values, units and sample rate stated. Simple, but long records get expensive.



2 Summarize first

Send RMS, peak, frequency from an FFT, not raw samples. Short, reliable, loses detail.



best default for lab work

3 Let it call tools

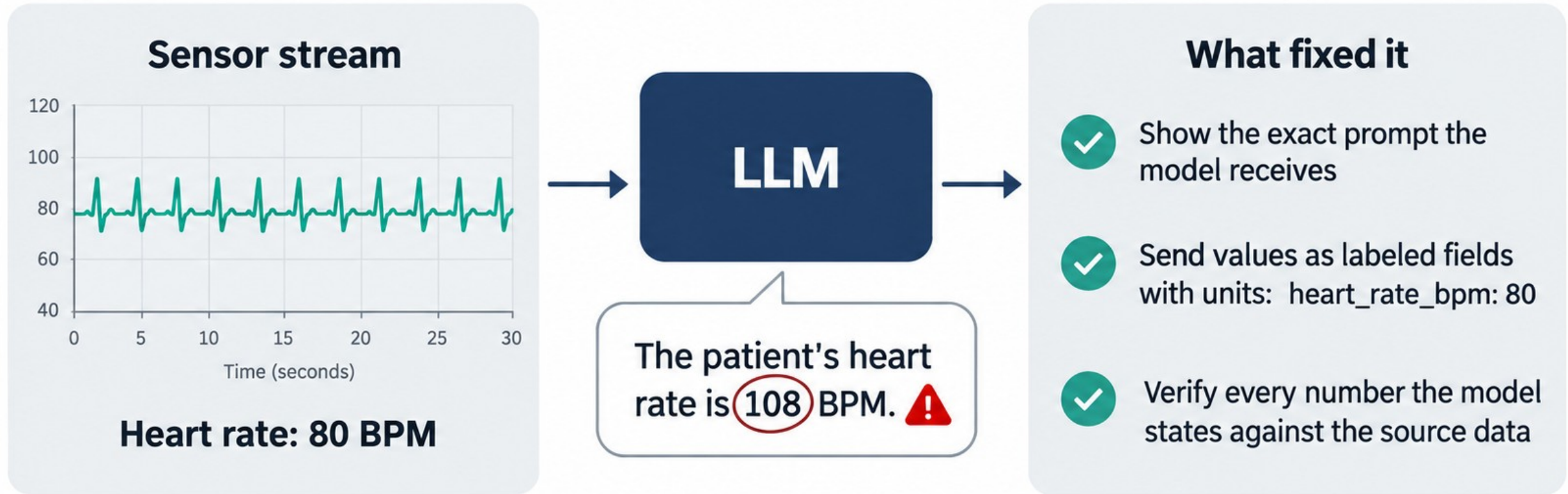
The LLM writes and runs code (NumPy, FFT, curve fit). Numbers stay exact; the LLM plans and explains.



4 Time-series foundation models

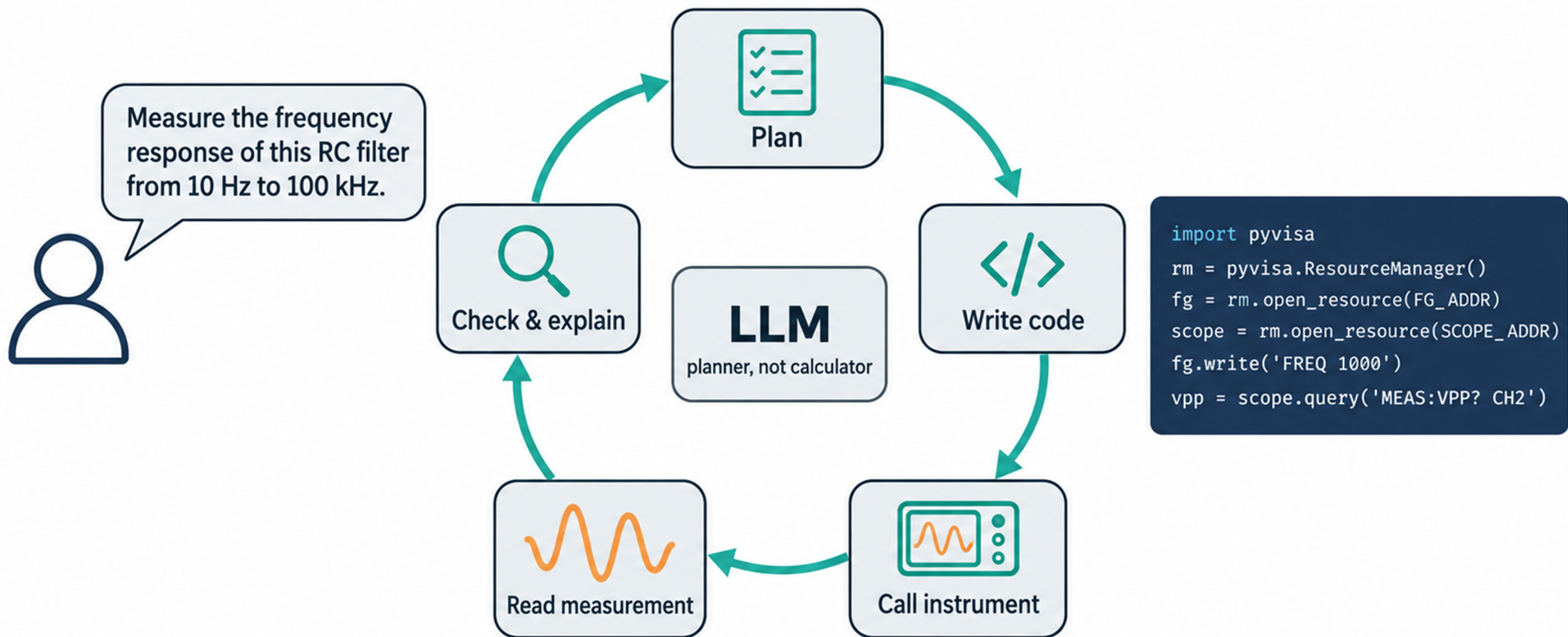
Values are scaled and quantized into bins, then modeled like tokens. Examples: Chronos (Amazon), TimesFM (Google).

Case study: when the numbers in the prompt disagree

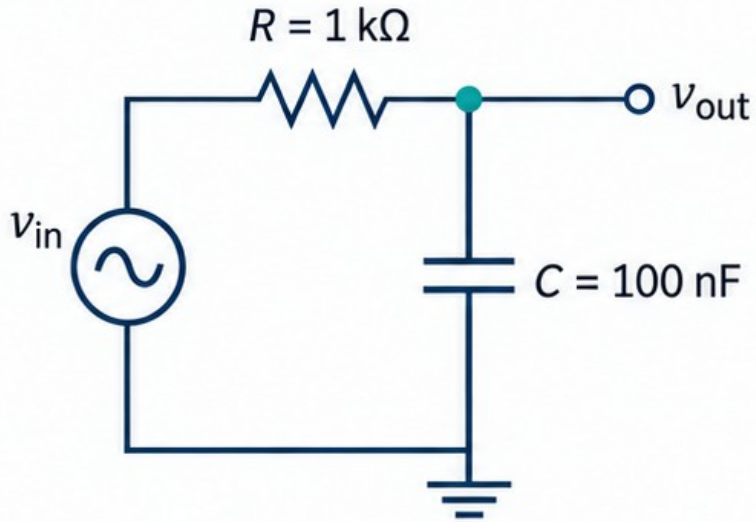


From an edge clinical-assistant prototype in our lab: the model read the sensor data but reported a different value.

LLMs that run instruments: the agent loop

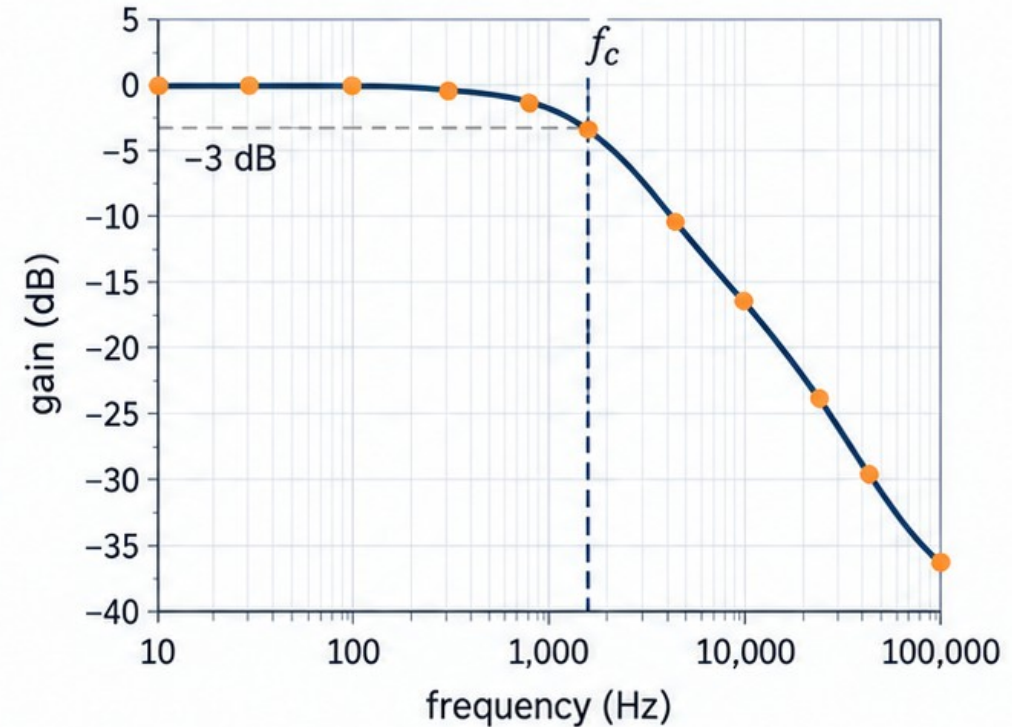


Example: an RC low-pass filter, measured by an agent



$$H(j\omega) = \frac{1}{1 + j\omega RC}$$

$$f_c = \frac{1}{2\pi RC} \approx 1.59\text{ kHz}$$



The agent sweeps the generator, reads the scope, fits f_c with code, then explains: **'Measured $f_c = 1.6\text{ kHz}$, within 1% of theory.'**

Where this fits with LabVIEW and your bench



Write the automation

- Generate Python / PyVISA or SCPI scripts from a plain-language test plan
- Explain an existing LabVIEW VI block by block
- Draft test procedures and checklists



Understand the data

- Summarize long logs and flag anomalies
- Suggest likely causes: grounding, aliasing, loading
- Turn results into a lab report draft



Keep humans in the loop

- Instrument limits enforced in code, never by the prompt
- Human approval before anything touches hardware
- Every reported number traced to a measurement

The LLM is the lab assistant that writes and explains. The instruments and the code do the measuring.

Pitfalls you will actually hit



Hallucinated numbers

States a value that is not in the data. Always verify against the source.



Unit and scale errors

mV vs V, ms vs s, dB vs linear. State units in every field.



Tokenization of digits

3.14159 is split into pieces, so arithmetic in text is unreliable.



Context limits

A 1 MS/s capture is far too long to paste. Summarize or use tools.

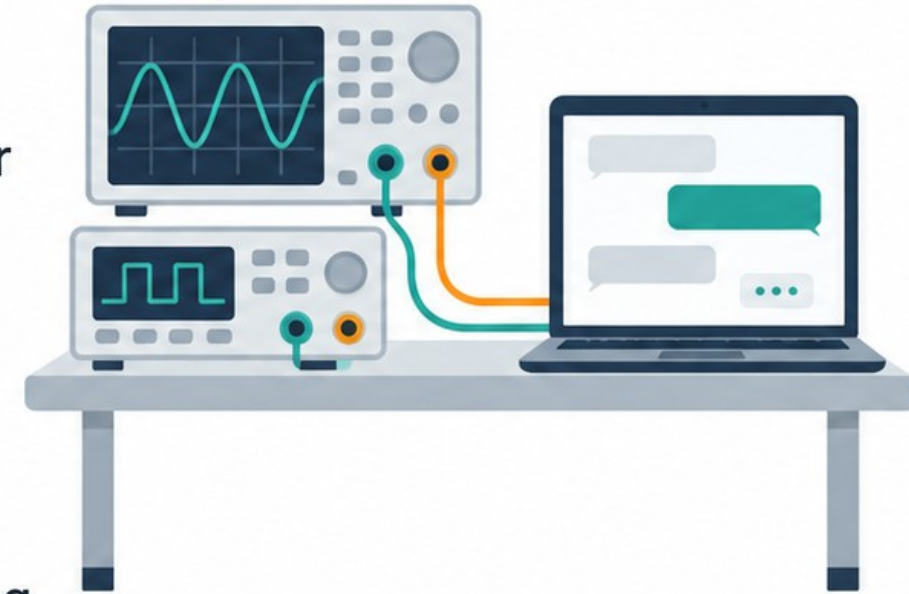


Unsafe commands

A wrong setpoint can damage hardware. Allowlists and hard limits in code.

A checklist for using LLMs with measurements

- ✓ Give units, sample rate and precision explicitly
- ✓ Let code do the math: FFT, fits, statistics
- ✓ Ask for code, then run and read it yourself
- ✓ Keep raw data out of the prompt; send summaries
- ✓ Log the exact prompt and the model's answer
- ✓ Cross-check every number against the instrument
- ✓ Enforce instrument limits in software
- ✓ A human approves anything that touches hardware

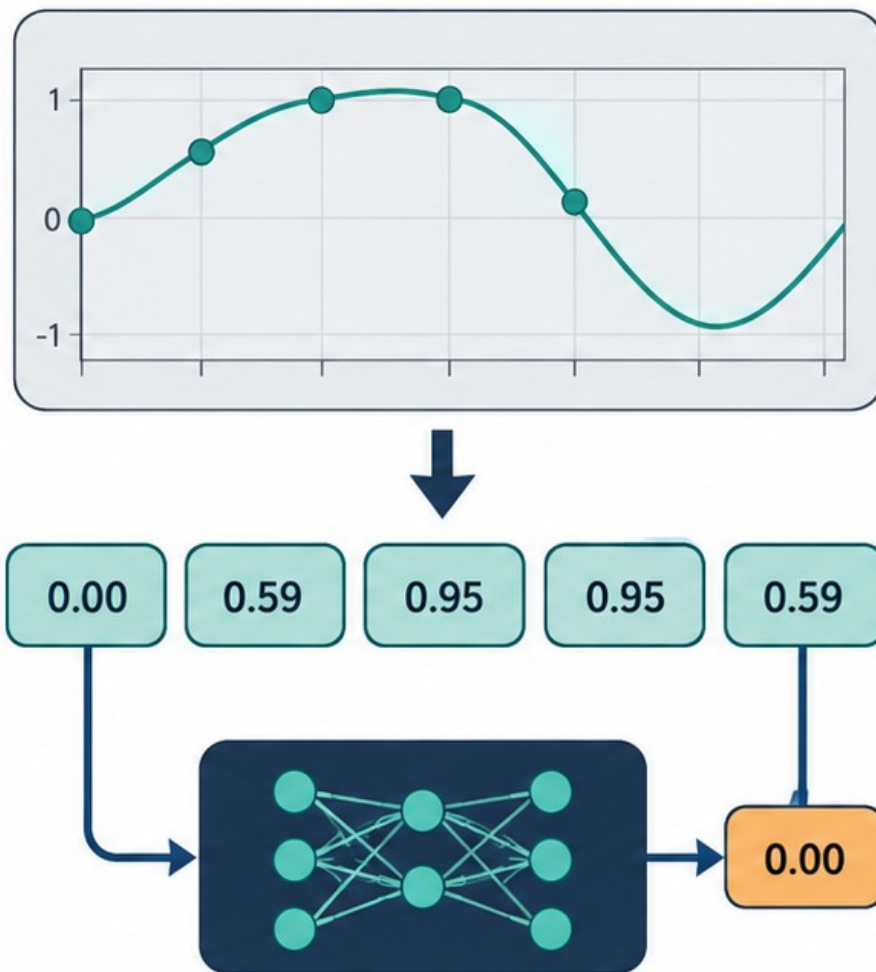


Summary

1 An LLM predicts the next token: $P(x_t | x_{<t})$

2 It sees your sine wave as text, not as volts and seconds

3 Best use in the lab: plan, write code, call instruments, explain. Let the tools do the measuring.

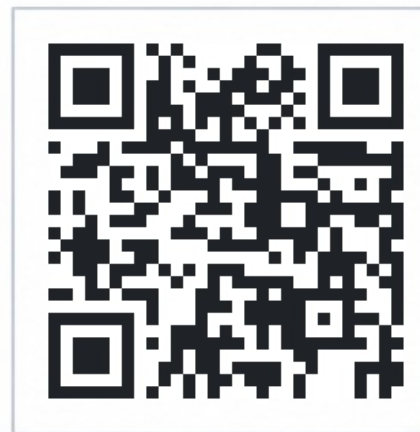


Questions?

Arash Ahmadi | arash.ahmadi@ou.edu

INQUIRE Lab, The University of Oklahoma

Join the LLM Engineering Club



Scan to join: inquirelab.ai/llm-club

Hands-on workshops: train and fine-tune LLMs on real GPUs

Free, open to every major, no AI background needed

